

Ingeniería de Datos II

Clase #1

11 de Marzo de 2026

Agenda

- Conocemos a los docentes y los estudiantes del curso.
- Que trata la materia y para que me servirá en la carrera.
- Consideraciones administrativas.
- Conceptos preliminares.
- Actividades en clase.
- Dudas y preguntas.

Nos presentamos



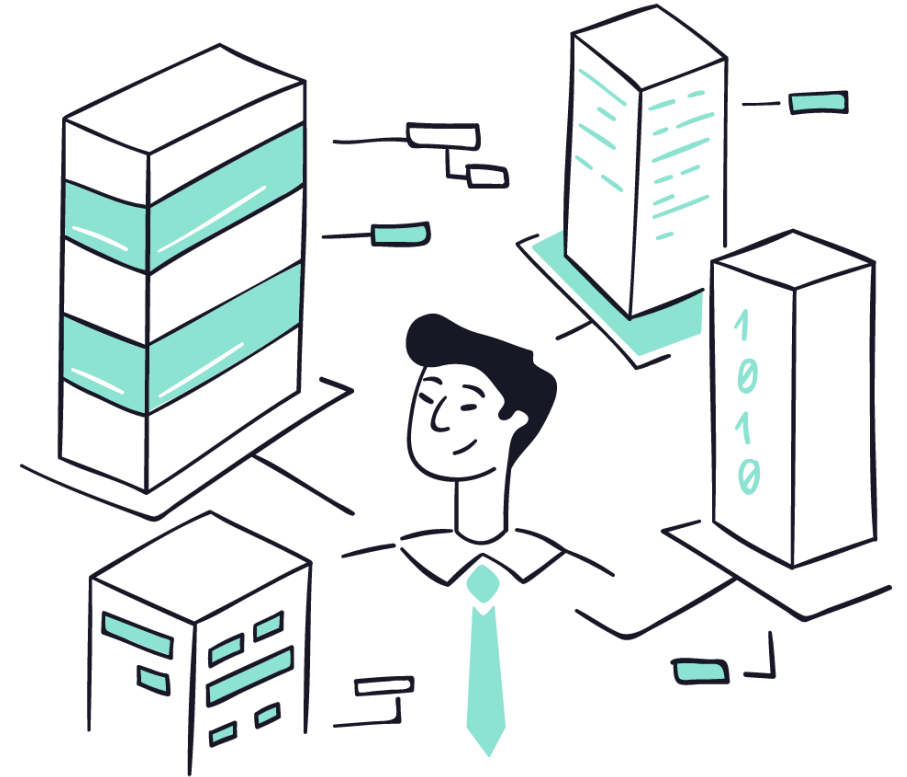
Ing. Hernán Novodvorski



<https://www.linkedin.com/in/hnovodvorski/>

Objetivos de la materia

- Comprender los fundamentos, principios y motivaciones de las bases de datos NoSQL en el contexto de sistemas modernos y distribuidos.
- Analizar los distintos modelos NoSQL (clave-valor, documentos, columnas y grafos) y sus casos de uso.
- Diseñar e implementar soluciones de almacenamiento y consulta de datos no relacionales, considerando escalabilidad, rendimiento y disponibilidad.
- Evaluar compromisos asociados al teorema CAP y a la consistencia en sistemas distribuidos.
- Aplicar buenas prácticas de modelado, consulta y gestión de datos NoSQL en escenarios reales de ingeniería de datos.



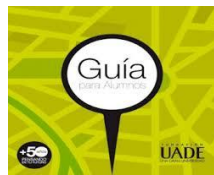


Consideraciones administrativas

- ¿Cómo se evaluará la materia?
 - Mediante la aprobación de dos exámenes parciales con nota igual o superior a 4 puntos y actividades prácticas en clase, contando con +75% de asistencia.
 - Los parciales se aprueban con 4 puntos que representan el 60% de las respuestas correctas.
 - Al aprobar los parciales junto con las actividades prácticas en clase, se estará en condiciones de rendir examen final regular.
 - Hay 1 posibilidad de recuperatorio para uno de los dos parciales que no sea aprobado en instancia normal.

http://www.dre.uade.edu.ar/guias/alumnos/guia_a.pdf

biblioteca@uade.edu.ar www.uade.edu.ar/biblioteca



Actividad

¿ Qué espero llevarme de la cursada de la materia ?



código 3492 1441

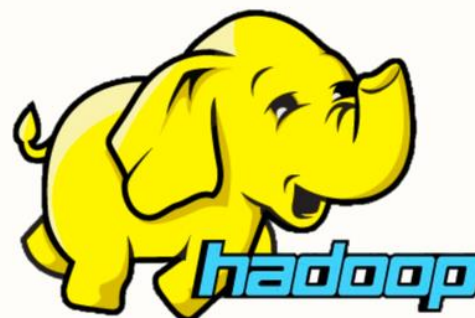
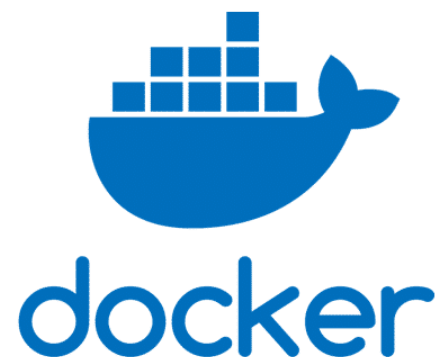
Bibliografía

- NoSQL data models trends and challenges (2018). Pivert, Olivier. ed. Laurent, Dominique. Laurent, Anne
- Next generation databases NoSQL, NewSQL, and Big Data (2015). Harrison, Guy.
- NoSQL for Mere Mortals (2015). Dan Sullivan.
- NoSQL Distilled (2013). Pramod J. Sadalage, Martin Fowler.



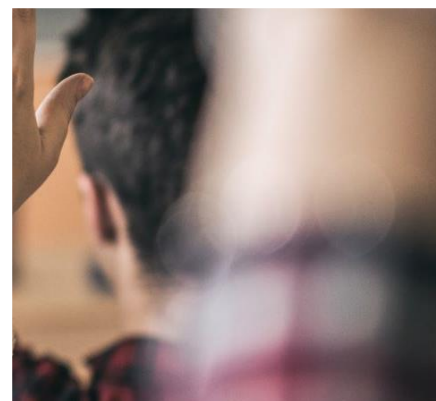
Se valorará ampliamente el trabajo de investigación individual de cada alumno mediante material complementario al visto en clase y a la bibliografía de la materia.

Algunas de las herramientas que estaremos usando durante la cursada



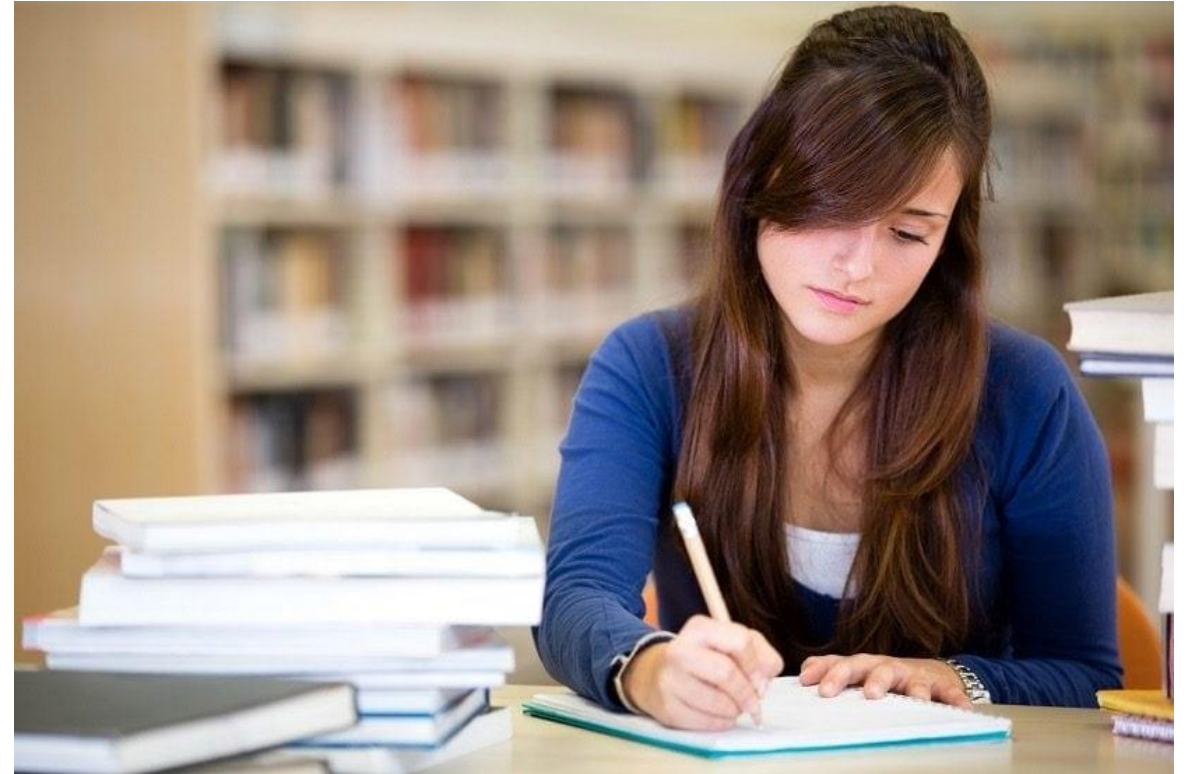


Dudas? Preguntas? Comentarios?





Repasando conceptos claves de Bases de Datos



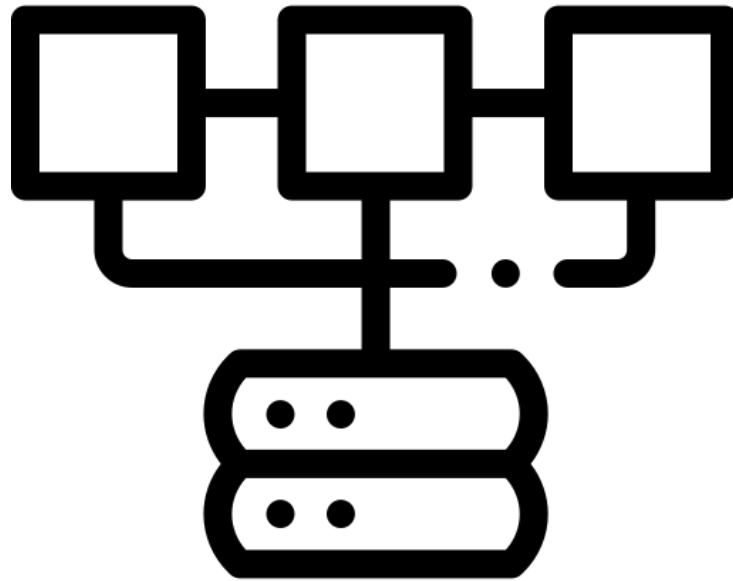
¿ Qué es una Base de Datos ?



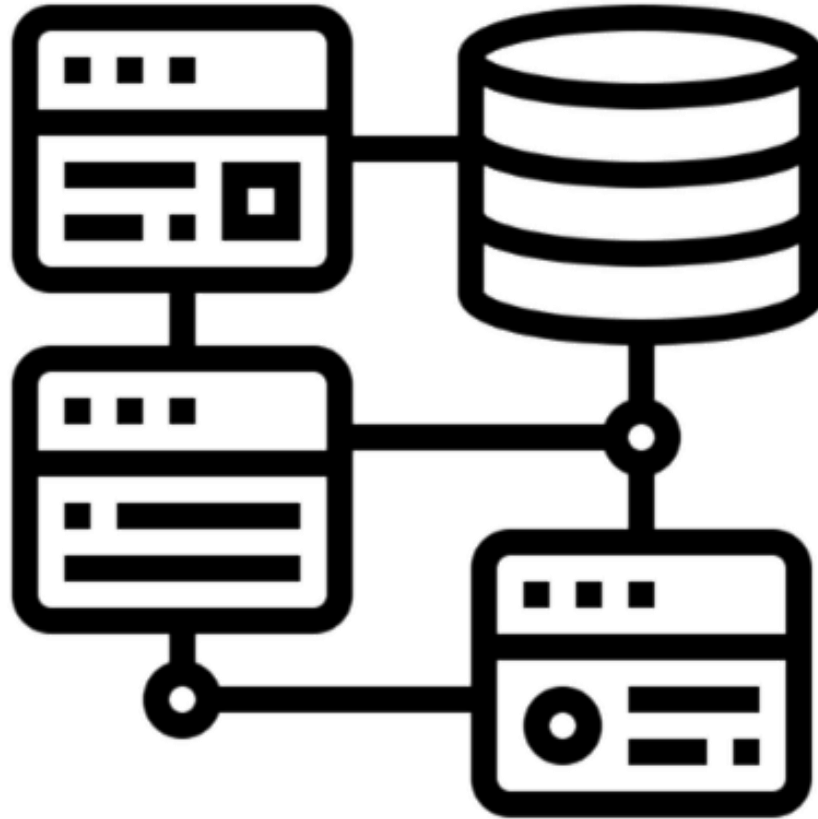
¿ Qué es un Sistema de Gestión de Base de Datos ?



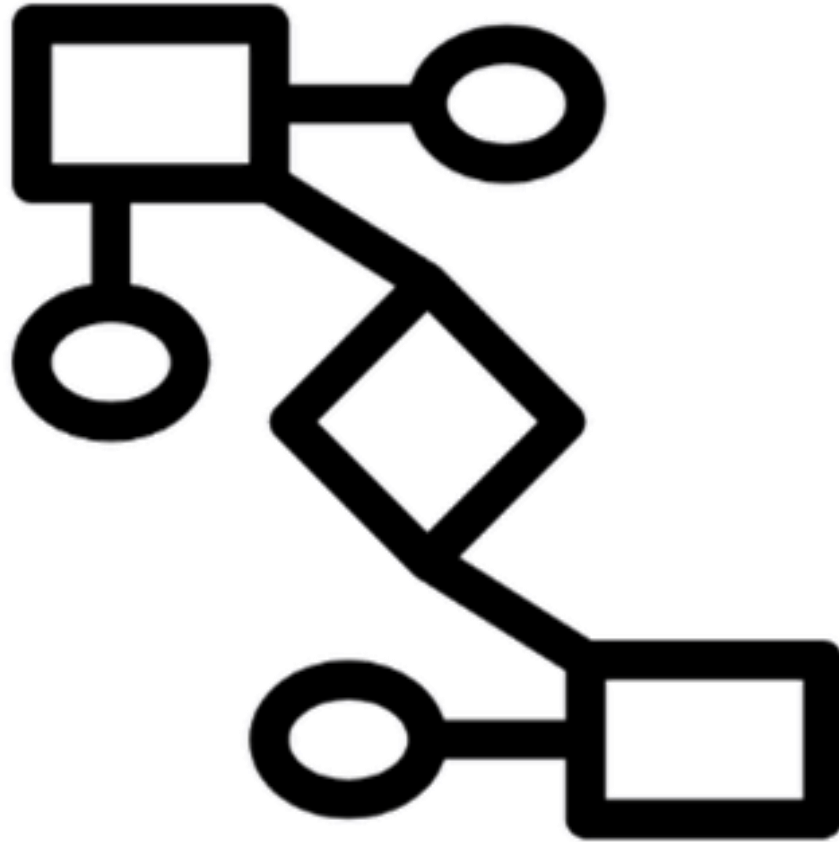
¿ Qué diferencia hay entre datos estructurados, semi estructurados y no estructurados? Dar ejemplos



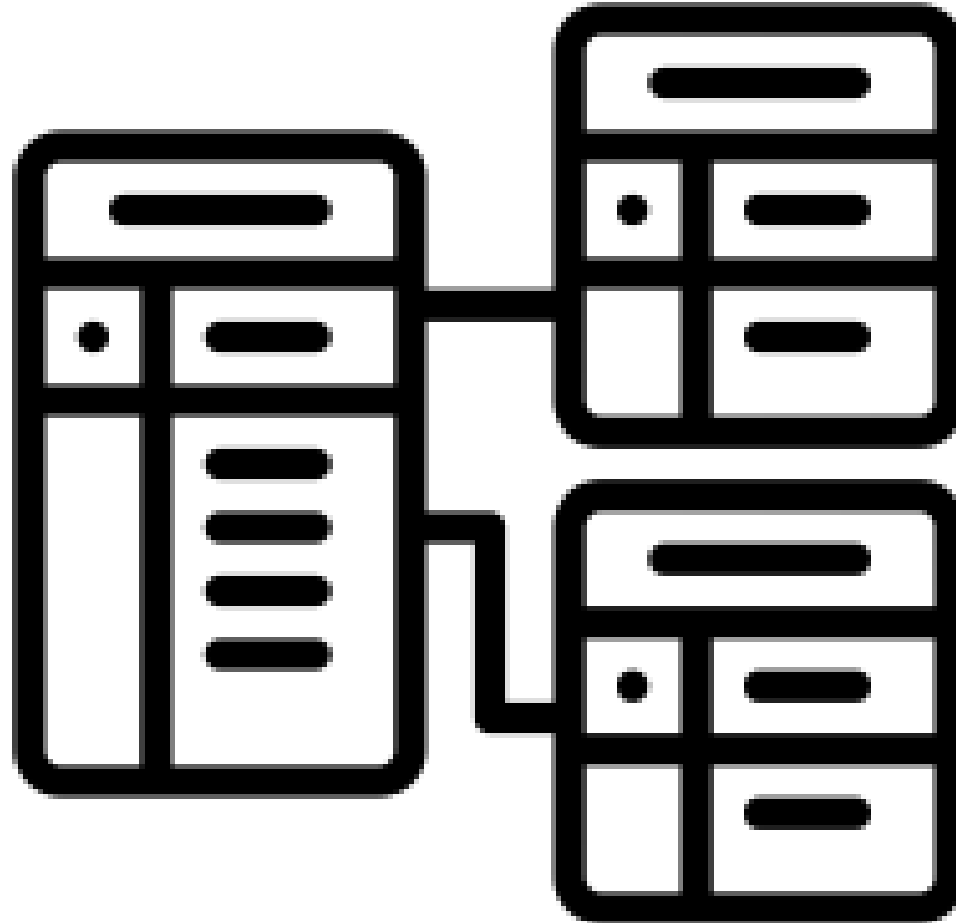
¿ Qué son las bases de datos relacionales?



¿ Qué es y para que sirve el modelado relacional?



¿ Qué es y para que sirve el modelado lógico?



¿ Qué es SQL y para que sirve?

SQL

¿ Qué nos dicen las propiedades ACID?

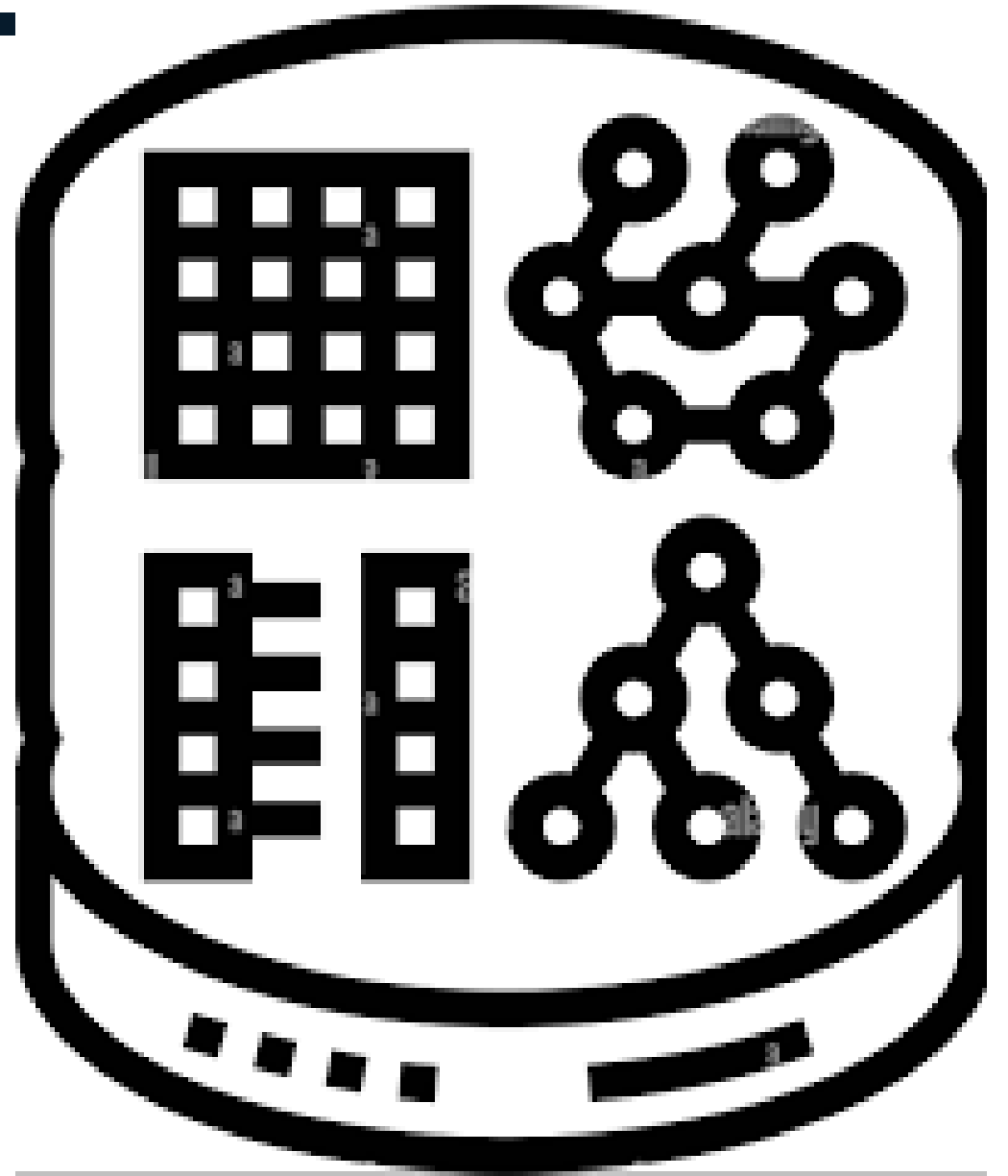
A C I D

¿Qué diferencia hay entre escalamiento horizontal y vertical?



Conceptos preliminares

Bases de datos no relacionales
(NOSQL)



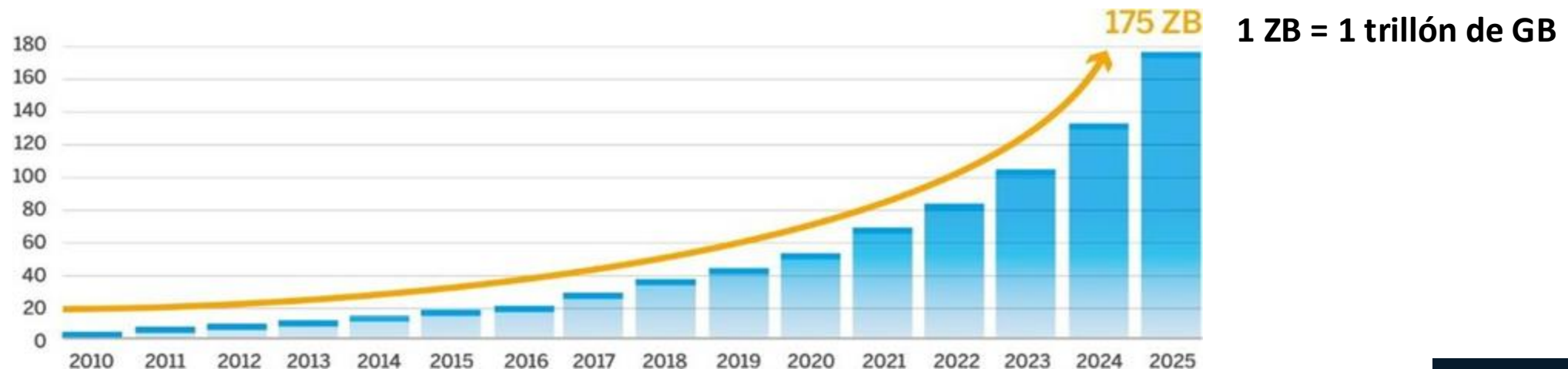
Características de las bases de datos relacionales

- Los datos se almacenan en columnas y tablas.
- Las relaciones se representan por datos.
- Lenguaje de manipulación de datos (DML).
- Lenguaje de definición de datos (DDL).
- Transacciones.
- Abstracción de la capa física.
- Las aplicaciones especifican el qué, no el cómo.
- La capa física puede cambiar sin cambiar las aplicaciones:
 - Creación de índices para soportar las consultas
 - Bases de datos en memoria

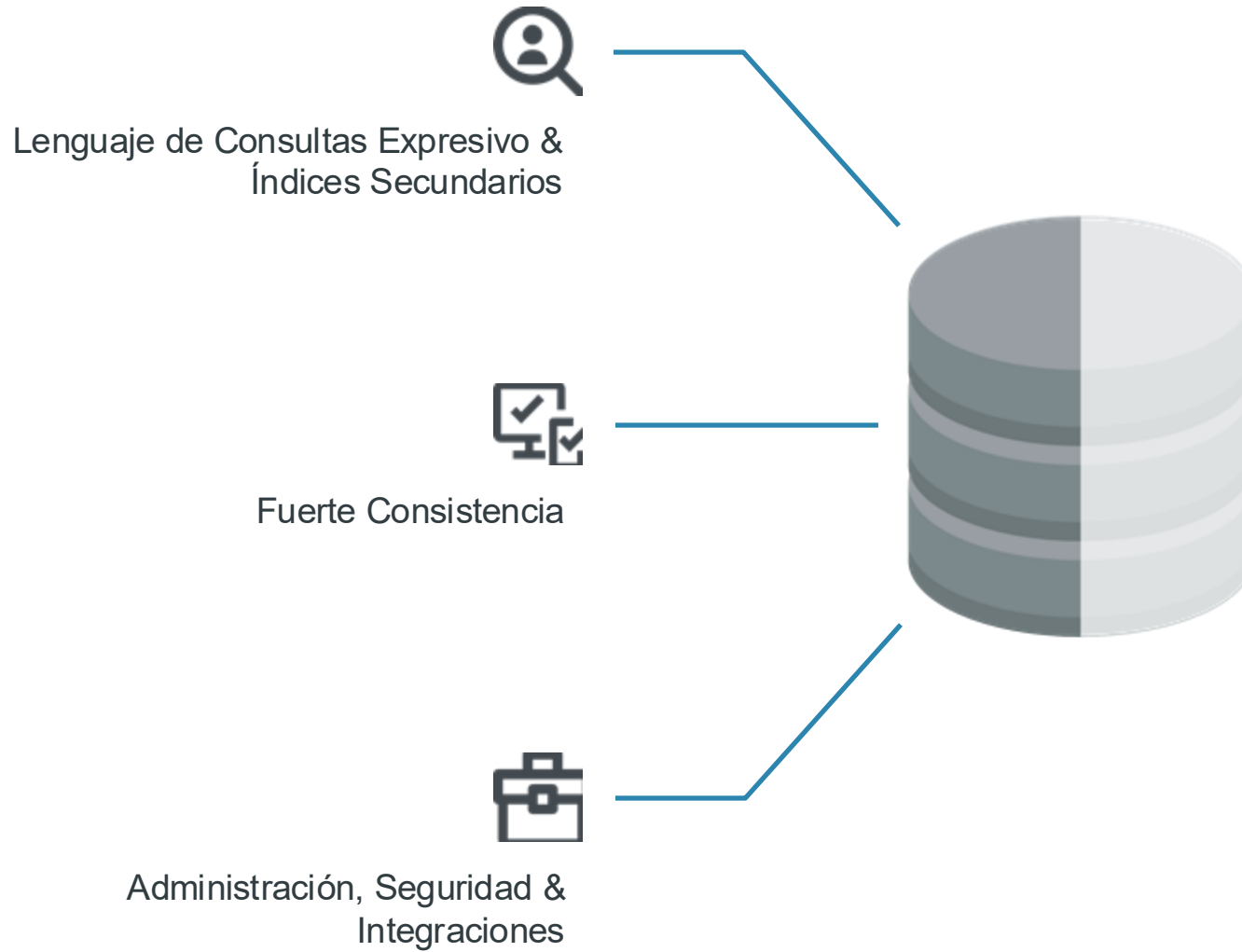


¿Por qué no SQL?

- Los sistemas de gestión de bases de datos relacionales (RDBMS) dominaron el mercado durante décadas.
- Después, con el auge de las grandes empresas tecnológicas como Apple, Facebook e Instagram, la esfera de datos mundial se multiplicó por 15 en la última década.
- Los RDBMS simplemente no estaban preparados para gestionar el nuevo volumen de datos ni los nuevos requisitos de rendimiento.



Relacional



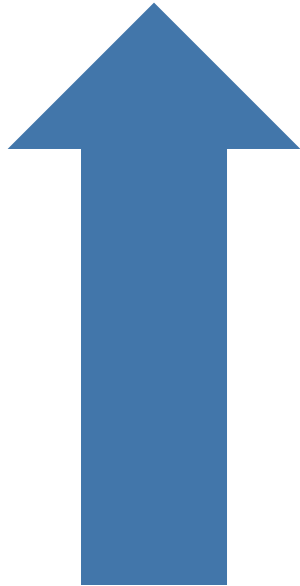
El mundo ha cambiado

Datos

Riesgo

Tiempo

Costos



No SQL



Lenguaje de Consultas Expresivo &
Índices Secundarios



Fuerte Consistencia



Administración, Seguridad &
Integraciones



Flexibilidad



Escalabilidad & Desempeño



Siempre Disponible,
Implementaciones Distribuidas

Ejemplo: Facebook

Estadísticas desde 2010



- 500 millones de usuarios
- 570 billones de vistas de páginas por mes
- 3 billones de fotos subidas por mes
- 1.2 millones de fotos servidas por mes
- 25 billones de piezas de contenido (actualizaciones, comentarios) compartidos cada mes
- 50 millones de operaciones del lado del server por segundo

2008: 10,000 servidores

2009: 30,000 servidores

...

Un sistema de bases de datos relacionales no sería suficiente para mantener esto funcionando!.

Ejemplo: Facebook

Arquitectura desde 2010



- **Cassandra**

- Sistema de almacenamiento distribuido NoSQL sin un único punto de falla.
- Para búsqueda en bandeja de entrada.



- **Hadoop / Hive**

- Sistema de almacenamiento masivo de datos.
- Implementación open source de MapReduce.
- Permite realizar cálculos en cantidades muy grandes de datos.
- Hive permite el uso de consultas SQL contra Hadoop

Ejemplo: Facebook

Arquitectura desde 2010 y posterior



- **Memcached**

- Sistema de cache de memoria distribuido.
- Capa de cache entre los servidores web y los servidores MySQL
 - Dado que el acceso es relativamente lento



- **HBase**

- Base de datos Hadoop, usada para e-mails, mensajería instantánea y SMS.
- Recientemente ha reemplazado a MySQL, Cassandra y algunos otros.
- Construida bajo el modelo de Google BigTable.

No SQL ?

- No SQL viene de:
 - No Relacional
 - No Sistema de Gestión de Base de Datos (SGBD)
 - No Solo SQL
- No SQL es un término 'paraguas' para todas las bases y almacenes de datos que no siguen los principios de los SGBD
 - Una clase de productos
 - Una colección de varios conceptos relacionados sobre almacenamiento y manipulación de datos
 - A menudo relacionado a grandes datasets

Los beneficios de No SQL

[\[https://www.mongodb.com/nosql-explained\]](https://www.mongodb.com/nosql-explained)

- En comparación con las bases de datos relacionales, **las bases de datos NoSQL son más escalables y ofrecen un rendimiento superior**, y su modelo de datos resuelve varios problemas para los que el modelo relacional no está diseñado:
 - Arquitectura distribuida geográficamente en lugar de una costosa arquitectura monolítica
 - Grandes volúmenes de datos estructurados, semiestructurados y no estructurados que cambian rápidamente.
 - Sprints ágiles, iteración rápida de esquemas y actualizaciones frecuentes de código.
 - Programación orientada a objetos, fácil de usar y flexible.

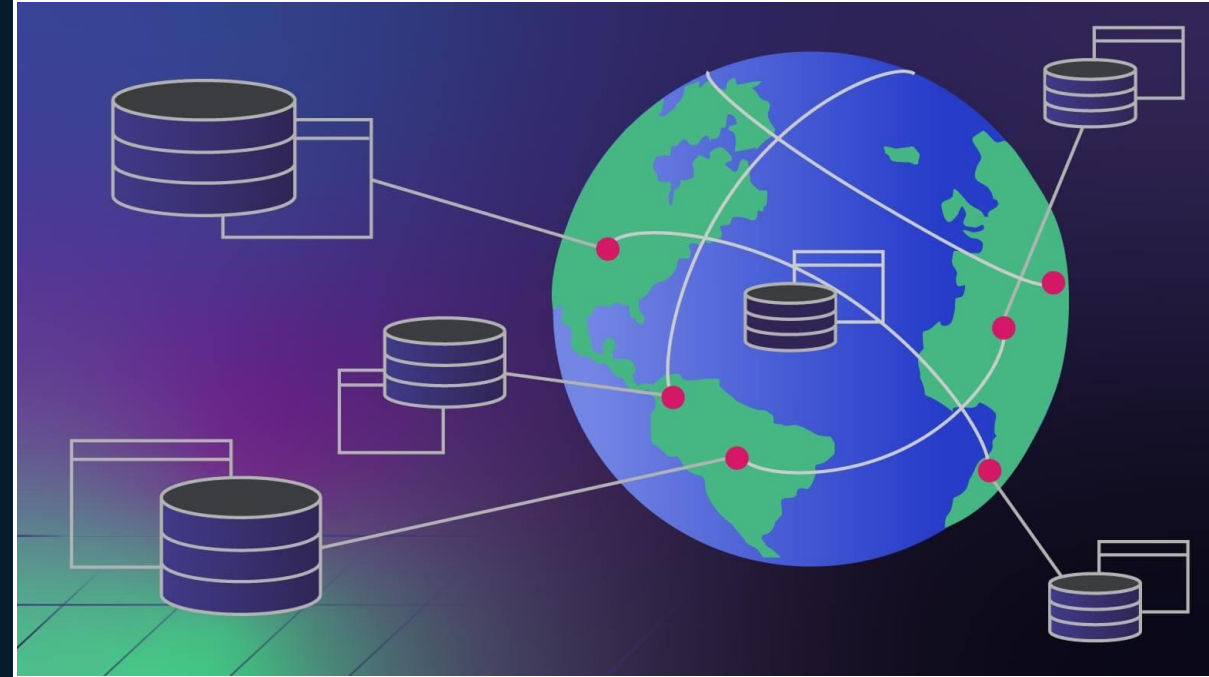
Escalamiento de bases de datos

<http://www.cs.tut.fi/~tjm/seminars/nosql2012/NoSQL-Intro.pdf>

- Las bases de datos relacionales se amplían agregando más hardware (escalamiento vertical).
- Las bases de datos no sql se amplían distribuyendo la carga, particionado (fragmentación) / replicación (escalamiento horizontal).

Conceptos preliminares

Distribución de datos



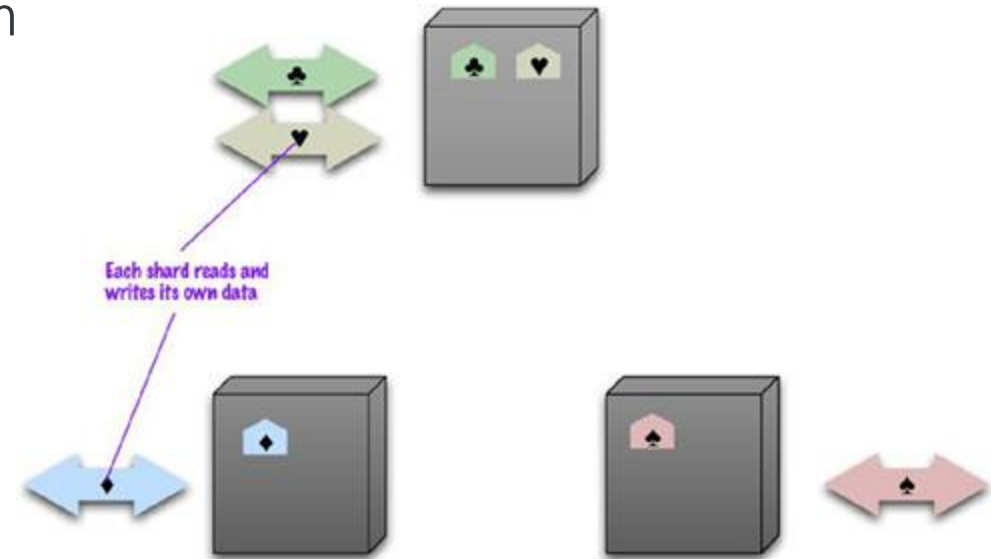
Distribución de datos

- En los sistemas No SQL los datos se distribuyen en grandes clusters.
- El agregado es una unidad natural para la distribución de datos.
- Modelos de distribución de datos:
 - En un único server (es una opción para algunas aplicaciones)
 - En múltiples servers
- Aspectos importantes de la distribución de datos:
 - **Fragmentación:** datos diferentes en nodos diferentes
 - **Replicación:** los mismos datos copiados sobre múltiples nodos

Distribución de datos

Fragmentación

- Diferentes partes de los datos en servidores diferentes
- **Caso ideal:** diferentes usuarios se comunican con diferentes nodos del servidor
- Los datos son accedidos en forma conjunta en el mismo nodo – ¡unidad agregada!
- **Ventajas**
 - Escalamiento horizontal de grandes volúmenes de datos
 - Latencia reducida
 - Optimiza el uso del almacenamiento
 - Puede mejorar tanto las lecturas como las escrituras
- **Desventajas:**
 - Mayor complejidad de diseño
 - Consultas que cruzan fragmentos son más costosas
 - Los clusters usan máquinas menos confiables – disminuye la resiliencia



Distribución de datos

Fragmentación – cont.

- Scatter / Gather:

- Es una forma costosa de ejecutar consultas distribuidas, donde la base debe preguntar a todos los nodos porque no sabe dónde está el dato.

- Hotspot:

- En sistemas distribuidos y bases de datos NoSQL, un hotspot es una concentración desbalanceada de carga (lecturas, escrituras o ambas) en uno o pocos nodos, mientras el resto del clúster está subutilizado.

Distribución de datos

Fragmentación – cont.

- Cuando usar fragmentación:
 - Big Data / grandes volúmenes
 - Sistemas con crecimiento constante
 - Bases NoSQL distribuidas
 - Casos donde la escalabilidad es crítica
- Ejemplos:
 - MongoDB (sharding)
 - Cassandra
 - DynamoDB
 - Hbase

Replicación de datos

Dos formas principales



maestro - esclavo



entre pares

Replicación maestro – esclavo

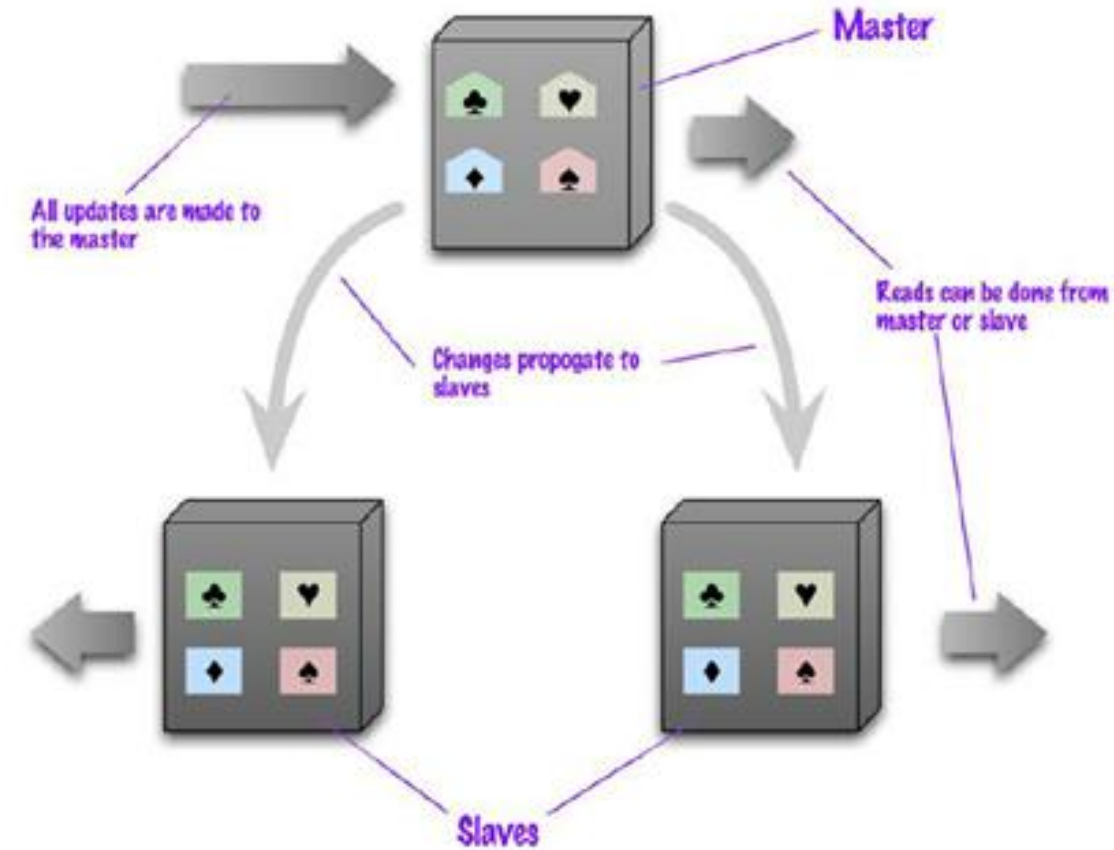
Principales características

- Maestro

- El maestro es la fuente autorizada de los datos
- Es el responsable de procesar cualquier actualización a los datos

- Esclavos

- Un proceso de replicación sincroniza los esclavos con el maestro
- Luego de una falla del maestro, se puede apuntar a que un esclavo sea nuevo maestro muy rápido



Replicación maestro – esclavo

Ventajas y desventajas

- Ventajas

- Más solicitudes de lectura:
 - Se adicionan más nodos esclavos
 - Se debe asegurar que todas las solicitudes de lectura se rutean a los esclavos
 - Si el maestro falla, los esclavos pueden seguir gestionando las peticiones de lectura.
 - Bueno para datasets de lectura intensiva

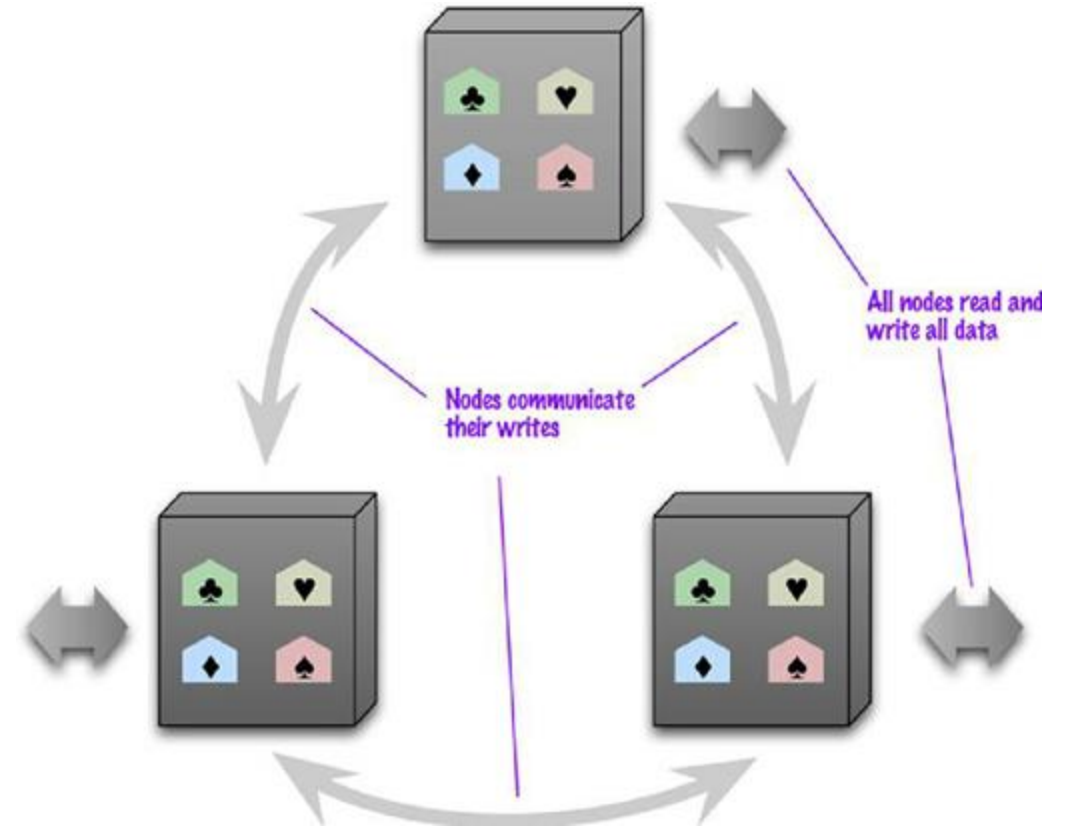
- Desventajas

- El maestro es un cuello de botella
 - Limitado por su capacidad para procesar actualizaciones y transmitirlas
 - Su falla elimina la capacidad de manejar escrituras hasta que:
 - Se restaure el maestro
 - Se nombre un nuevo maestro
 - Inconsistencias debido a propagación lenta de los cambios a los esclavos
 - Malo para datasets con tráfico pesado de escritura

Replicación entre pares

Principales características

- Todas las réplicas tienen el mismo peso, todas pueden aceptar escrituras.
- La pérdida de cualquiera de los nodos no previene del acceso a los datos.



Replicación entre pares

Ventajas y desventajas

- **Ventajas**

- Se puede superar los fallos de los nodos sin perder el acceso a los datos
- Se pueden agregar fácilmente nodos para mejorar la performance

- **Desventajas**

- Inconsistencia
- Propagación lenta de los cambios a copias en diferentes nodos
- Las inconsistencias en la lectura causan problemas, pero son relativamente transitorias.
- Dos personas pueden actualizar al mismo tiempo copias diferentes de un mismo registro almacenado en nodos distintos: un conflicto de escritura-escritura.
 - Las escrituras inconsistentes son eternas.

Distribución de datos

Replicación

- Cuando usar replicación:
 - Sistemas críticos que no pueden caerse
 - Aplicaciones con muchas lecturas
 - Sistemas geográficamente distribuidos
 - Alta disponibilidad y recuperación ante fallos
- Ejemplos:
 - PostgreSQL / MySQL (replication)
 - MongoDB (replica sets)
 - Redis (replicas)
 - Neo4j (cluster)

Consistencia

- Es el mayor cambio de una base de datos relacional centralizada a un modelo de clusters No SQL
 - Las bases de datos relacionales tienen una fuerte consistencia
 - Los sistemas No SQL mayormente tienen consistencia eventual



Conflictos de lectura y escritura

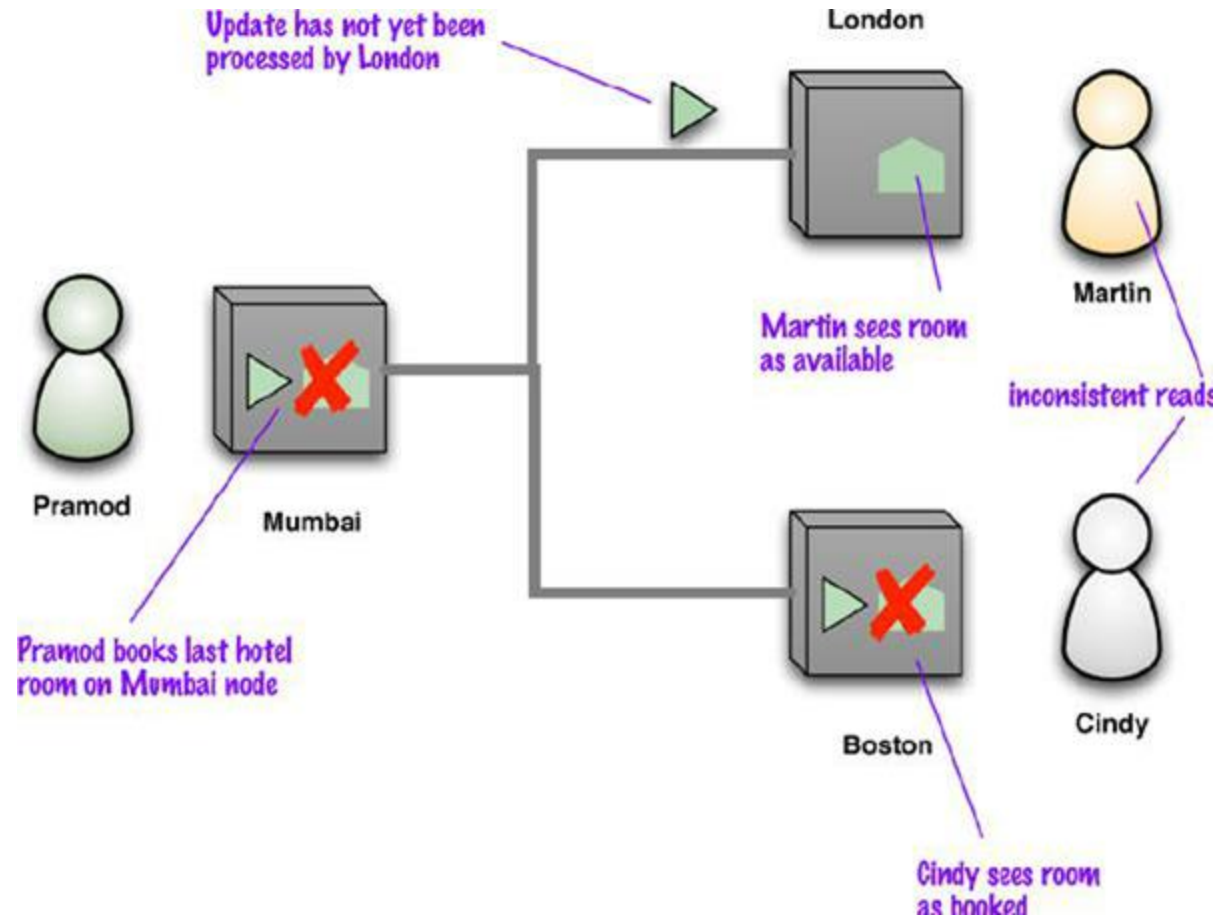
Resumen

- **Lectura – lectura**
 - Personas diferentes ven datos diferentes al mismo tiempo.
 - Datos obsoletos: no actualizados.
- **Escritura – escritura**
 - Dos (o más personas) actualizando los mismos ítems de datos al mismo tiempo;
 - Si el servidor las serializa: una se aplica e inmediatamente es sobrescrita por la otra (actualización perdida).
- **Lectura – escritura**
 - Una lectura en el medio de dos escrituras lógicamente relacionadas.

Conflictos de lectura

Ejemplo

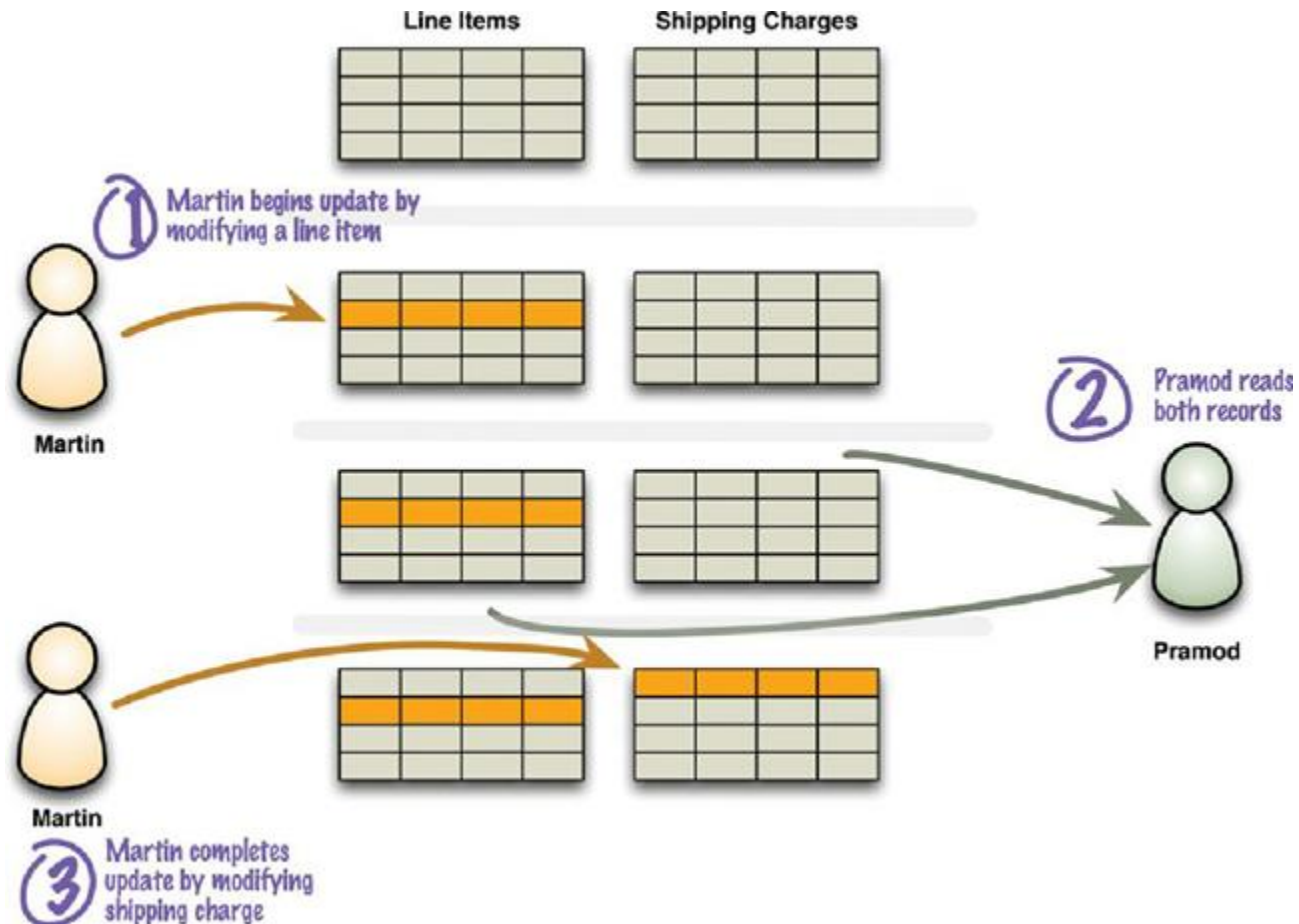
- La replicación es fuente de inconsistencia:



Conflictos de lectura – escritura

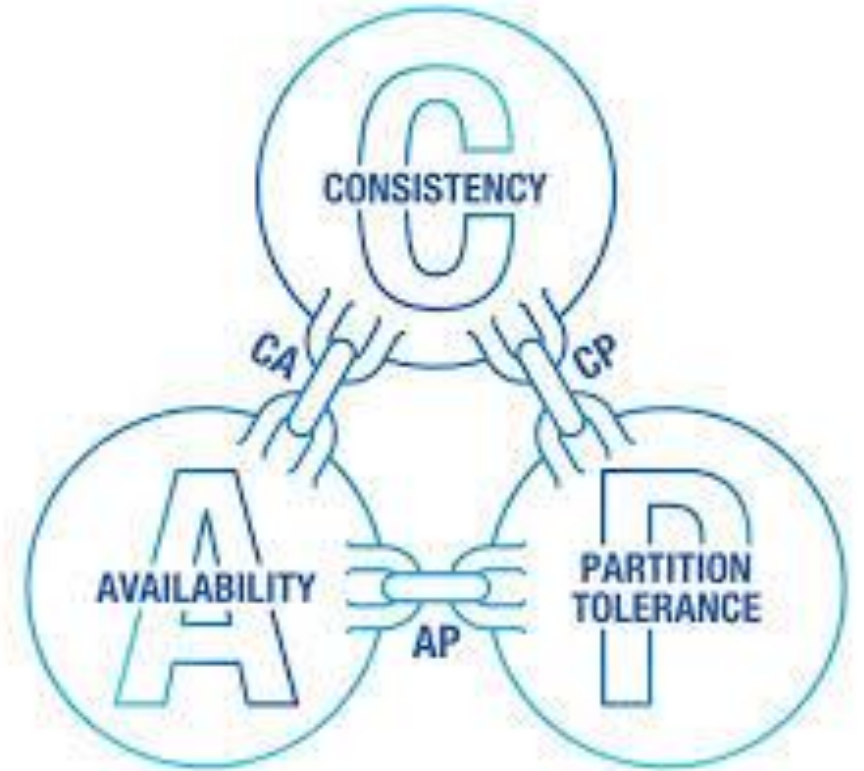
Ejemplo

- Una lectura en el medio de dos escrituras lógicamente relacionadas:



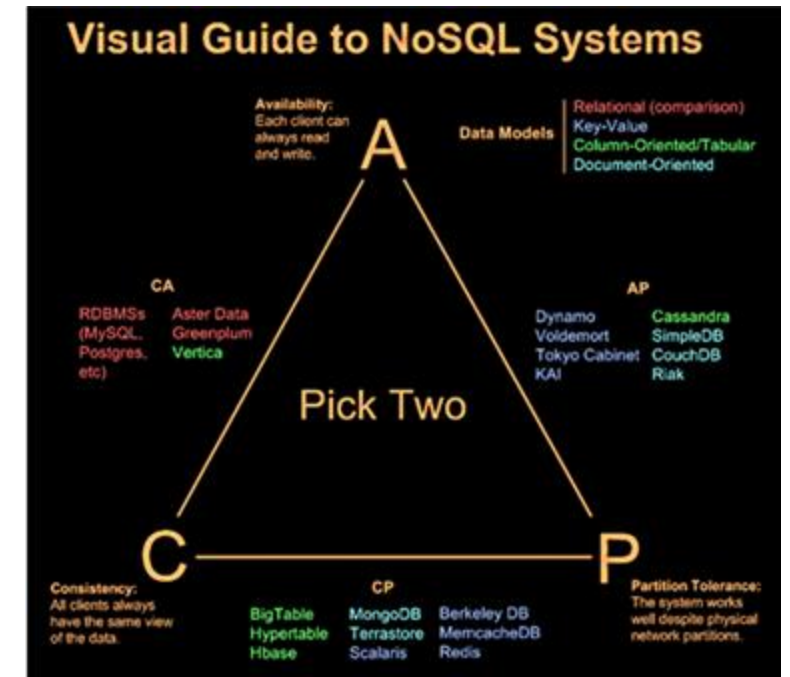
Conceptos preliminares

Teorema CAP



Teorema CAP

- Proporciona una forma congruente y lógica de evaluar los problemas que plantea la garantía del tipo ACID en los sistemas distribuidos.
- Como máximo se pueden garantizar dos de las siguientes tres características a la vez, pero no las tres:
- **Consistencia**
 - Cada cliente tiene la misma visión de los datos
- **Disponibilidad**
 - Cada cliente siempre puede leer y escribir
- **Tolerancia a las particiones**
 - El sistema funciona bien a pesar de las particiones de las redes físicas



Fragmentación: prioriza AP

Replicación: síncrona prioriza CP, asíncrona prioriza AP

Propiedades BASE

- El teorema CAP prueba que es imposible garantizar estricta Consistencia y Disponibilidad mientras se es capaz de ser tolerante a las particiones de red.
- Esto lleva a bases de datos con garantías ACID relajadas.
- En particular, tales bases de datos aplican las propiedades BASE:
 - **Básicamente Disponible (Basically Available)**: el sistema garantiza la disponibilidad;
 - **Estado Blando (Soft State)**: el estado del sistema podría cambiar con el tiempo;
 - **Consistencia Eventual (Eventual Consistency)**: el sistema eventualmente se convertirá en consistente.



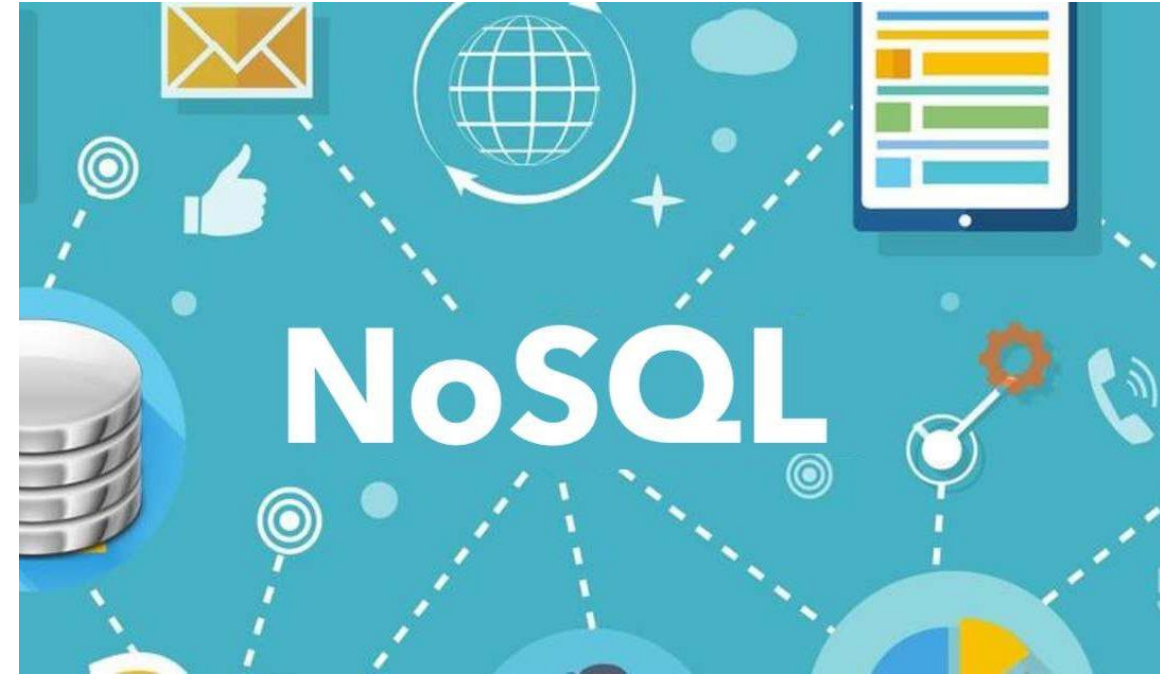
Practiquemos un poco ...



Joinmyquiz.com 27596190

Conceptos preliminares

Principales bases de datos NoSQL



Principales bases de datos NoSQL

- Bases de datos documentales
- Bases de datos de grafos
- Bases de datos de clave valor
- Bases de datos columnares

Bases de datos documentales

Bases de datos documentales

Principales características

- Los datos se almacenan como documentos (generalmente en formatos JSON, BSON o XML).
- Cada documento representa una entidad completa (por ejemplo: un cliente, un pedido).
- Los documentos pueden contener estructuras anidadas y colecciones internas.
- No requieren un esquema fijo.
- Cada documento puede tener campos distintos.
- Permiten evolucionar el modelo de datos sin migraciones complejas.
- Son ideales para aplicaciones en constante cambio

Relational

Person:

Pers_ID	Surname	First_Name	City
0	Miller	Paul	London
1	Ortega	Alvaro	Valencia
2	Huber	Urs	Zurich
3	Blanc	Gaston	Paris
4	Bertolini	Fabrizio	Rom

Car:

Car_ID	Model	Year	Value	Pers_ID
101	Bentley	1973	100000	0
102	Rolls Royce	1965	330000	0
103	Peugeot	1993	500	3
104	Ferrari	2005	150000	4
105	Renault	1998	2000	3
106	Renault	2001	7000	3
107	Smart	1999	2000	2

no relation

MongoDB Document

```
{
  first_name: 'Paul',
  surname: 'Miller'
  city: 'London',
  location: [45.123,47.232],
  cars: [
    { model: 'Bentley',
      year: 1973,
      value: 100000, ... },
    { model: 'Rolls Royce',
      year: 1965,
      value: 330000, ... }
  ]
}
```

Bases de datos documentales

Principales usos

- **Aplicaciones web y móviles:** perfiles de usuario, configuraciones, sesiones.
 - Ej.: Meta (Facebook), Airbnb.
- **E-commerce y catálogos:** productos con atributos variables, carritos, pedidos.
 - Ej.: Amazon, eBay.
- **Gestión de contenidos (CMS):** artículos, noticias, metadata flexible.
 - Ej.: The New York Times, BBC.
- **Microservicios:** persistencia desacoplada por servicio.
 - Ej.: Netflix, Uber.
- **Big Data operativo:** eventos, logs, telemetría.
 - Ej.: LinkedIn, Twitter/X.
- **Gestión de identidades:** usuarios, roles, permisos.
 - Ej.: Google, Microsoft.

Bases de datos documentales

Ejemplo de formato BSON (JSON binario)

```
[ { "_id": ObjectId("60e4b0fb8ed5a00072382b07"),
  "name": "John Doe", "age": 30, "email":
  "johndoe@example.com", "address": { "street": "123
  Main St", "city": "Anytown", "state": "CA",
  "zipcode": "12345" }, "interests": ["hiking", "reading",
  "photography"]
},
{ "_id":
  ObjectId("60e4b13f8ed5a00072382b08"), "name": "Jane
  Smith",
  "age": 25, "email": "janesmith@example.com",
  "address": {"street": "456 Oak St", "city": "Sometown",
  "state": "NY", "zipcode": "54321"
},
  "interests": ["traveling", "painting", "gardening"]
},
```

```
{
  "_id":
  ObjectId("60e4b15e8ed5a00072382b09"),
  "name": "Alice Johnson",
  "age": 35, "email":
  "alicejohnson@example.com",
  "address": {"street": "789 Elm St", "city":
  "Anothertown",
  "state": "TX", "zipcode": "67890"
},
  "interests": ["cooking", "knitting", "running"]
}
```

Bases de datos documentales

Bases más conocidas

-  **mongoDB.** • Base de datos de documentos que ofrece una gran escalabilidad y flexibilidad, y un modelo de consultas e indexación avanzado.
-  • **Amazon DynamoDB:** servicio de base de datos noSQL ofrecido por Amazon como parte de Amazon Web Services.
-  **Couchbase** • Proyecto de código abierto, paquete de software distribuido multi-modelo orientado a base de datos documental y que está optimizado para aplicaciones interactivas.
-  • **CouchDB:** gestor de bases de datos de código abierto de Apache, cuyo foco está puesto en la facilidad de su uso y en ser "una base de datos que asume la web de manera completa".
-  **Cloud Firestore** • **Firebase Firestore:** base de datos NoSQL flexible, escalable y en la nube, creada en la infraestructura de Google Cloud, a fin de almacenar y sincronizar datos para el desarrollo tanto del lado del cliente como del servidor..

Bases de datos documentales

Principales ventajas y desventajas

- **Ventajas**

- Esquema flexible
- Modelado natural de datos
- Buen rendimiento en actualización de datos (CRUD)
- Escalabilidad horizontal mediante particionado
- Alta integración con aplicaciones modernas

- **Desventajas**

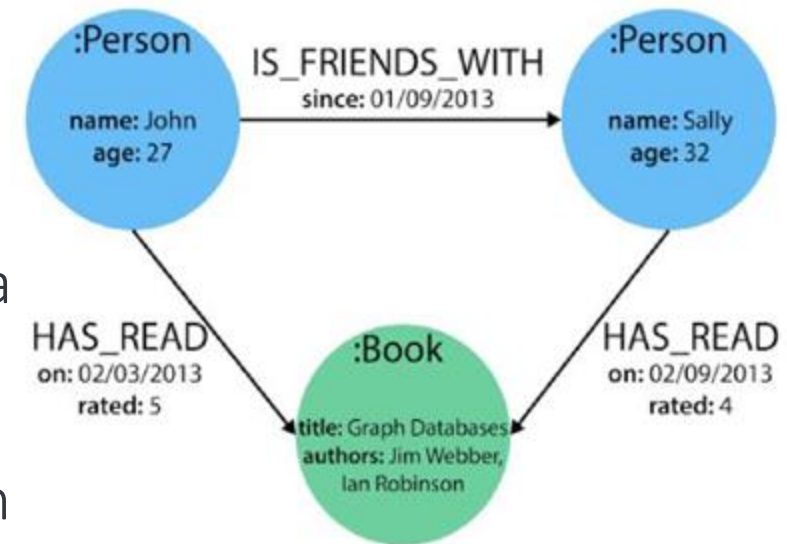
- Limitaciones en relaciones complejas
- Redundancia de datos
- Consistencia limitada en algunos escenarios
- Consultas analíticas menos eficientes
- Menor estandarización

Bases de datos de grafos

Bases de datos de grafos

Principales características

- Los datos se representan como nodos (entidades) y relaciones (aristas).
- Tanto nodos como relaciones pueden tener propiedades.
- El modelo prioriza las conexiones entre los datos.
- No requieren un esquema rígido.
- Se pueden agregar nuevos tipos de nodos, relaciones o propiedades sin rediseñar la base.
- Facilitan la evolución del modelo de datos.
- Ideal para dominios complejos y cambiantes.
- Las relaciones están almacenadas explícitamente, no calculada tiempo de consulta.
- Evitan joins costosos típicos de bases relacionales.
- Excelente rendimiento al recorrer múltiples niveles de relacion
- Optimizadas para consultas relacionales complejas, no para transacciones simples masivas.



Bases de datos de grafos

Principales usos

- **Redes sociales:** relaciones entre personas, seguidores, interacciones.
 - Ej.: Meta (Facebook), LinkedIn.
- **Sistemas de recomendación:** productos, contenidos o usuarios relacionados.
 - Ej.: Amazon, Netflix.
- **Detección de fraude:** análisis de patrones sospechosos y conexiones indirectas.
 - Ej.: PayPal, Mastercard.
- **Gestión de identidades y accesos (IAM):** usuarios, roles, permisos y jerarquías.
 - Ej.: Microsoft, Google.
- **Knowledge Graphs:** representación de conocimiento y relaciones semánticas.
 - Ej.: Google (Search), Amazon (Alexa).
- **Análisis de dependencias:** software, redes, logística, supply chain.
 - Ej.: Uber, Airbnb.

Bases de datos de grafos

Bases más conocidas



- Software libre de base de datos orientada a grafos, implementado en Java..



Amazon Neptune

- Base de base de datos de grafos administrados publicado por Amazon. Se utiliza como un servicio web y forma parte de Amazon Web Services.



ArangoDB

- Sistema de base de datos multimodelo, admite tres modelos de datos con un núcleo de base de datos y un lenguaje de consulta unificado AQL.



- **Azure Cosmos DB:** servicio de base de datos multimodelo distribuido globalmente ofrecido por Microsoft.



- Base de datos de grafos distribuida de código abierto bajo la Fundación Linux.

Bases de datos de grafos

Principales ventajas y desventajas

- **Ventajas**

- Excelente manejo de relaciones complejas
- Alto rendimiento en consultas relacionales
- Modelo de datos intuitivo
- Consultas expresivas
- Esquema flexible
- Soporte para algoritmo de grafos

- **Desventajas**

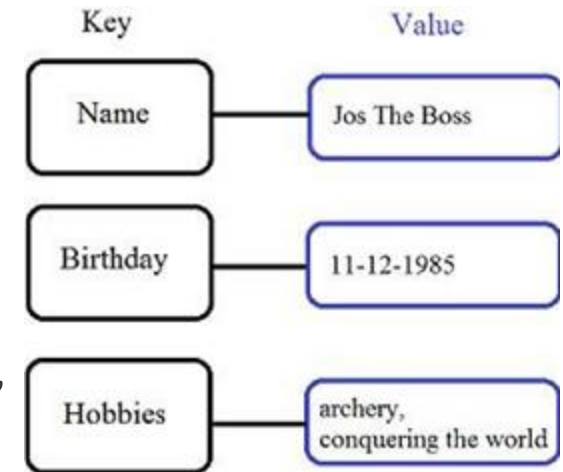
- No óptimo para datos poco relacionados
- Escalabilidad distribuida más compleja
- Menor rendimiento en operaciones simples masivas
- Curva de aprendizaje mayor
- Menor estandarización

Bases de datos de clave valor

Bases de datos de clave valor

Principales características

- Los datos se almacenan como pares clave única → valor.
- La clave es el único medio de acceso al dato.
- Alto rendimiento.
- El valor puede ser simple (string, número) o complejo (JSON, binario), pero es opaco para la base.
- No existe esquema ni estructura interna reconocida por el motor.
- La base no interpreta el contenido del valor.
- Máxima flexibilidad, mínima semántica.
- Ideales para datos simples y de acceso directo.
- Escalabilidad horizontal nativa.



Bases de datos de clave valor

Bases más conocidas



- Motor de base de datos en memoria, basado en el almacenamiento en tablas de hashes (clave/valor) pero que opcionalmente puede ser usada como una base de datos durable o persistente.



- Base de datos incrustada de alto rendimiento para datos de clave-valor.



- Base de datos distribuida NoSQL clave-valor con replicación local avanzada y multi-cluster que garantiza lecturas y escrituras incluso en caso de fallos de hardware o particiones de red.



- Base de datos fiable distribuida de clave-valor para los datos más críticos de un sistema distribuido, con un enfoque en ser: simple, segura, rápida y confiable.



- Base de almacenamiento rápido de código abierto basada en pares clave-valor con almacenamiento en disco. Está escrita por Jeffrey Dean y Sanjay Ghemawat de Google, que también escribieron parte de la plataforma Google

Bases de datos de clave valor

Principales usos

- **Caché de aplicaciones:** almacenamiento temporal de datos frecuentemente accedidos.
 - Ej.: Netflix, Twitter/X (uso intensivo de Redis).
- **Gestión de sesiones de usuario:** sesiones web, tokens, estados temporales.
 - Ej.: Amazon, Shopify.
- **Configuraciones y feature flags:** parámetros de aplicaciones en tiempo real.
 - Ej.: LinkedIn, Uber.
- **Contadores y métricas en tiempo real:** likes, vistas, rankings, rate limiting.
 - Ej.: Facebook / Meta, Instagram.
- **Colas simples y mensajería liviana:** procesamiento rápido de eventos.
 - Ej.: Slack, GitHub.
- **IoT y datos efímeros:** estados de dispositivos, lecturas temporales.
 - Ej.: AWS (DynamoDB), Google Cloud.

Bases de datos de clave valor

Ejemplo

Crear una base de datos clave-valor utilizando un diccionario en Python

```
key_value_db = {}
```

Agregar datos a la base de datos

```
key_value_db["producto:1"] = {"nombre": "Laptop", "precio": 1200, "stock": 10}
```

```
key_value_db["producto:2"] = {"nombre": "Teléfono", "precio": 800, "stock": 20}
```

```
key_value_db["producto:3"] = {"nombre": "Tableta", "precio": 500, "stock": 15}
```

Acceder a los datos utilizando las claves

```
producto_1 = key_value_db["producto:1"]
```

```
print("Datos del producto 1:", producto_1)
```

```
producto_2 = key_value_db["producto:2"]
```

```
print("Datos del producto 2:", producto_2)
```

Bases de datos de clave valor

Principales ventajas y desventajas

- **Ventajas**

- Simplicidad del modelo
- Muy alto rendimiento y baja latencia
- Escalabilidad horizontal nativa
- Alta disponibilidad y tolerancia a fallos
- Bajo overhead operativo

- **Desventajas**

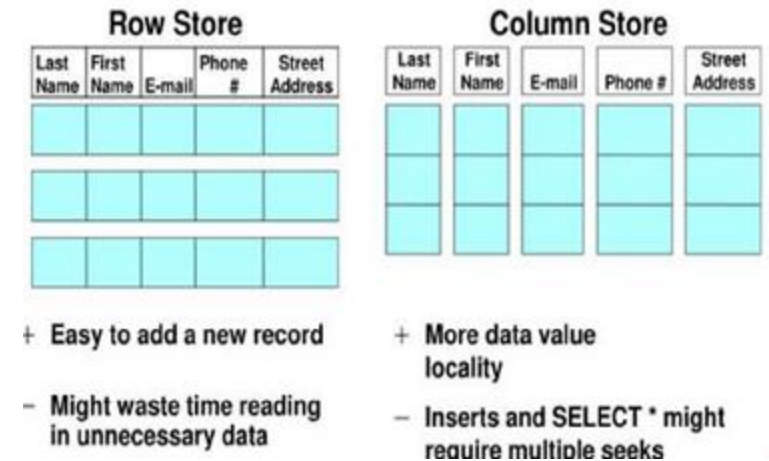
- Capacidad de consulta muy limitada
- Falta de semántica del dato
- No aptas para relaciones entre datos

Bases de datos columnares

Bases de datos columnares

Principales características

- Los datos se almacenan por columnas y no por filas.
- Cada columna se guarda de forma contigua en disco o memoria.
- Optimiza el acceso a conjuntos específicos de atributos, no a registros completos.
- Ideal para escenarios de lectura analítica.
- Requieren un esquema definido (columnas y tipos).
- Menor flexibilidad que documentales o grafos.
- Cambios de esquema suelen ser más costosos.
- Optimizadas para lecturas intensivas.



Bases de datos columnares

Principales usos

- **Business Intelligence (BI) y reporting:** consultas agregadas, KPIs, dashboards.
 - Ej.: Google, Microsoft (analítica interna y reporting).
- **Data Warehouses / Data Analytics:** análisis histórico de grandes volúmenes de datos.
 - Ej.: Amazon (Redshift), Netflix (analítica de uso y comportamiento).
- **Análisis de logs y eventos:** monitoreo, auditoría, métricas operativas.
 - Ej.: Uber, Cloudflare.
- **Big Data y analítica a gran escala:** procesamiento analítico distribuido.
 - Ej.: Facebook / Meta, LinkedIn.
- **Análisis financiero y de riesgos:** agregaciones complejas sobre datos históricos.
 - Ej.: Goldman Sachs, Visa.
- **Machine Learning y analítica avanzada:** preparación y exploración de datos.
 - Ej.: Spotify, Airbnb.

Bases de datos columnares

Ejemplo

Ejemplo de tabla en base de datos relacional

Id_usuario	Nombre	Edad	Género	Ubicación
123456	Flavia Díaz	30	Femenino	Concordia
789012	Héctor López	25	Masculino	Tres Arroyos
345678	Daiana Romero	27	Femenino	Rosario

Ejemplo de datos organizados en columnas

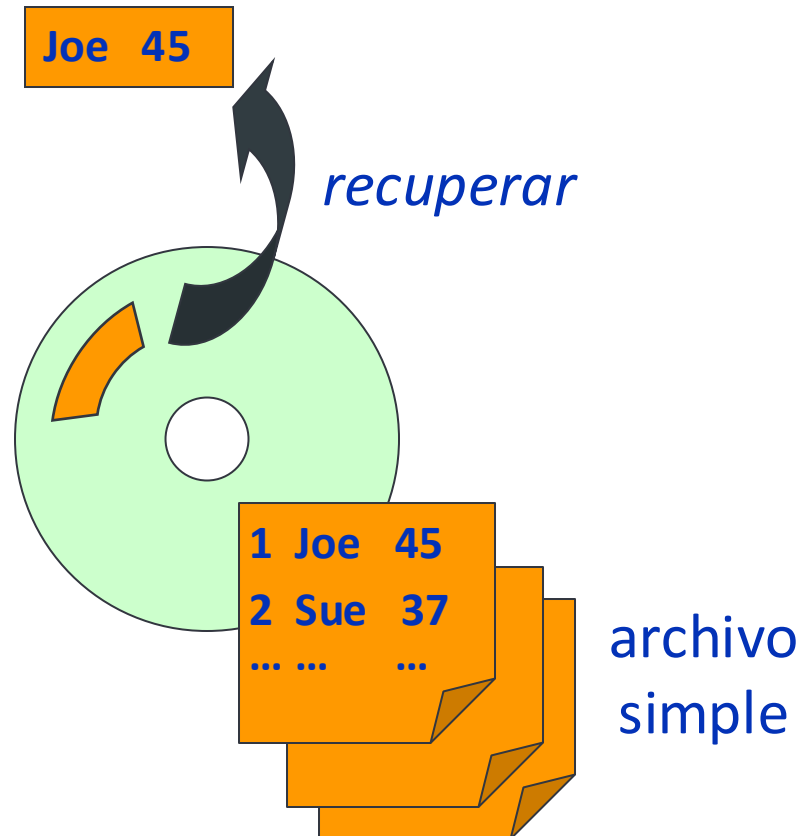
Id_usuario	123456	789012	345678
Nombre	Flavia Díaz	Héctor López	Daiana Romero
Edad	30	25	27
Género	Femenino	Masculino	Masculino
Ubicación	Concordia	Tres Arroyos	Rosario

- Este formato permite una consulta y recuperación eficientes de atributos específicos en muchas filas.
- Por ejemplo, si se quisiera recuperar las edades de todos los usuarios, sólo tendría que acceder a la columna «Edad».
- Del mismo modo, si se desea obtener toda la información sobre un usuario concreto (identificado por su ID de usuario), deberá acceder a las columnas correspondientes a dicho usuario.

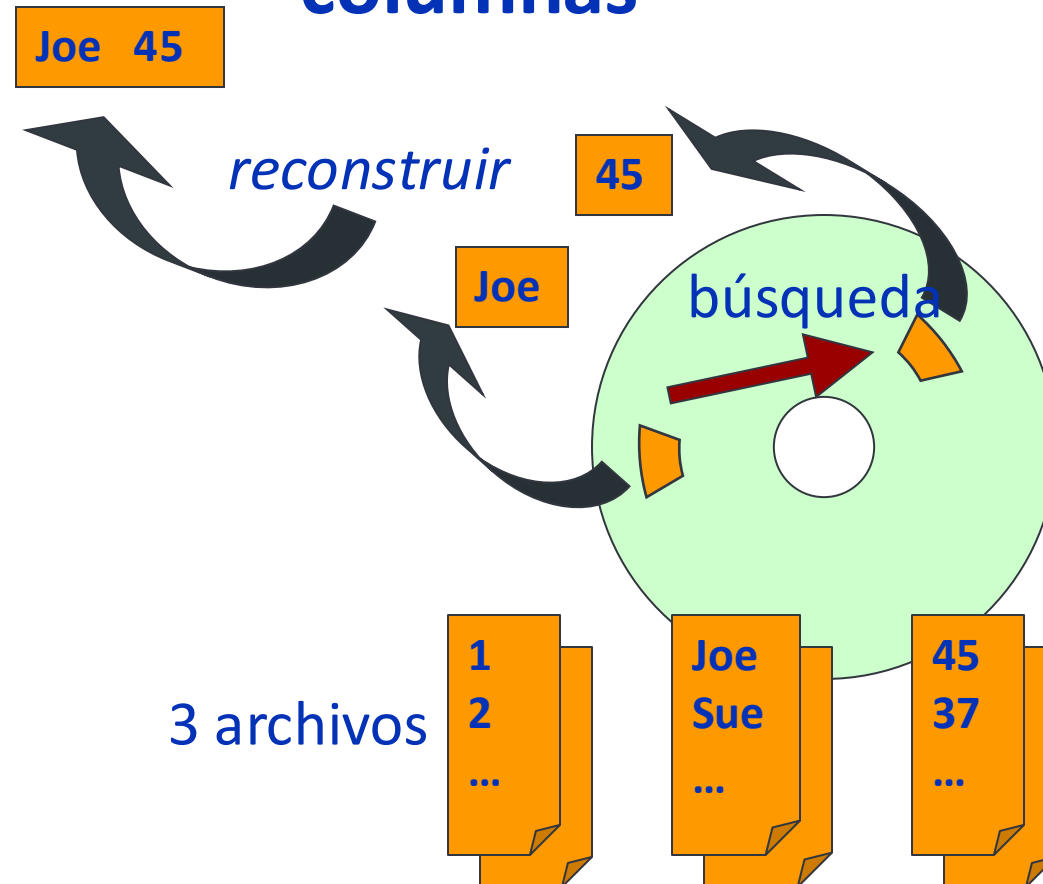
Bases de datos columnares

Ejemplo – cont.

Almacenamiento en filas



Almacenamiento en columnas



Bases de datos columnares

Bases más conocidas



- **Amazon Redshift:** forma parte de Amazon Web Services (AWS), se usa para análisis de big data.



- Fork de MySQL de código abierto combina base de datos columnar y relacional.



- Base de datos de SAP que también combina base de datos columnar y relacional.



- Es gratuito, está basado en Apache Hadoop y escrito en JAVA.



- Es de código abierto desarrollado con foco en la minería de datos.

Bases de datos columnares

Principales ventajas y desventajas

- **Ventajas**

- Alto rendimiento en consultas analíticas
- Lectura eficiente de datos
- Alta compresión de datos
- Escalabilidad para análisis masivo
- Ideales para BI y reporting

- **Desventajas**

- Bajo rendimiento en operaciones transaccionales (OLTP)
- Menor flexibilidad del esquema
- Latencia más alta para accesos por fila
- No adecuadas como base operativa principal
- Mayor complejidad en pipelines de datos



Practiquemos un poco ...